

Compiler Design (CS24301) — Full Course Master Book

Complete End-to-End B.Tech CSE 5th Semester Study Book & PYQ Bank

EXECUTIVE MASTER STUDY GUIDE & PYQ BANK

Compiler Design (CS24301)

Birla Institute of Technology, Mesra | B.Tech CSE 5th Semester (NEP 2024–25 Scheme)

Complete Course Structure & Syllabus Matrix

Module	Core Syllabus Scope	Key Algorithms & Formulations
Module I	Lexical Analysis & Automata	6 Phases, Two-Buffer Scheme, Thompson RE to NFA, Subset Construction, Hopcroft DFA Minimization
Module II	Syntax Analysis & Parsers	CFG, Left Recursion/Factoring, FIRST & FOLLOW, LL(1) Table, Shift-Reduce, LR(0), SLR(1), CLR(1), LALR(1)
Module III	Semantic Analysis & Types	Syntax-Directed Definitions (SDD), Synthesized/Inherited, S/L-Attributed, Type Checking, Symbol Tables
Module IV	Intermediate Code & Runtime	Three-Address Code (Quadruples/Triples), Multi-D Array Addressing, Backpatching, Activation Records
Module V	Optimization & Code Gen	Basic Blocks, CFG, DAG, Loop Invariant Code Motion, Reaching Definitions, Register Coloring

Exam Preparation & High-Yield Guide

This publication-grade master book consolidates all 5 modules with formal mathematical proofs, KaTeX-rendered formulas, step-by-step worked traces on classic datasets, and comprehensive model answers to BIT Mesra end-semester examination questions.

Module I: Lexical Analysis & Automata Theory — Complete Syllabus Topics

1. Language Processors (Compiler, Interpreter, Assembler, Preprocessor)
2. 6 Phases of Compiler Architecture with End-to-End Running Example
3. Compiler Front-End vs Back-End & Pass Structures
4. Lexical Analysis Role, Tokens, Patterns & Lexemes
5. Input Buffering (Two-Buffer Scheme & Sentinels)
6. Regular Expressions & Regular Definitions for Programming Tokens
7. Thompson's Construction Algorithm (RE to NFA Step-by-Step)
8. Subset Construction Algorithm (NFA to DFA Transition Table)
9. Hopcroft's DFA State Minimization Algorithm (Equivalence Partitioning)
10. Lex & Flex Lexical Analyzer Generator Specifications

Topic 1 & 3: Language Processing Systems & Compiler Architecture

A **Compiler** is a sophisticated system software that translates a computer program written in a high-level source language (e.g., C, C++, Java) into an equivalent target program in low-level machine code or assembly language, while detecting and reporting diagnostic errors.



Figure 1.1: Context of a Compiler in Language Processing Pipeline

System Tool	Input → Output	Primary Responsibilities & Core Functions
1. Preprocessor	Raw Source Code → Pure Source Code	Macro expansion (`#define`), file inclusion (`#include`), conditional compilation (`#ifdef`), stripping comments.
2. Compiler	Pure Source Code → Target Assembly	Translates high-level syntax into assembly language, performs type checking, syntax validation, intermediate code generation, and optimization.
3. Assembler	Assembly Code → Relocatable Object Code	Translates mnemonic assembly instructions into binary machine opcodes and resolves memory offsets.
4. Linker / Loader	Object Files + Libs → Absolute Executable	Linker: Resolves external cross-file symbol references and library routines. Loader: Loads executable binary into primary RAM, sets up stack/heap memory spaces, and initializes CPU instruction pointer.

Topic 2: The 6 Phases of a Compiler (End-to-End Walkthrough)

A compiler operates as a sequence of distinct analytical and synthesis phases. Every phase transforms the source program from one intermediate representation to another, interacting continuously with the **Symbol Table** and **Error Handler**.



Complete Running Example: Step-by-Step Translation of Statement `position = initial + rate \* 60`

1. Phase 1: Lexical Analysis (Scanning):

Reads raw input character stream and converts into a stream of tokens:

$\langle \text{id}, 1 \rangle \langle = \rangle \langle \text{id}, 2 \rangle \langle + \rangle \langle \text{id}, 3 \rangle \langle * \rangle \langle \text{num}, 60 \rangle$

Symbol table stores: `1: position (float)`, `2: initial (float)`, `3: rate (float)`.

2. Phase 2: Syntax Analysis (Parsing):

Constructs a hierarchical Syntax Tree enforcing operator precedence (* over +):

```
      =
     / \
  id1   +
     / \
  id2   *
     / \
  id3  60
```

3. Phase 3: Semantic Analysis (Type Checking):

Checks static semantic rules, ensuring operand type compatibility. Inserts implicit type conversion node `inttofloat(60)`:

```
      =
     / \
  id1   +
     / \
  id2   *
     / \
  id3  inttofloat(60)
```

4. Phase 4: Intermediate Code Generation (ICG):

Generates machine-independent Three-Address Code (TAC) with at most one operator per instruction:

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

5. Phase 5: Code Optimization:

Performs compile-time constant folding and eliminates temporary variables:

```
t1 = id3 * 60.0    -- Constant folded: inttofloat(60) → 60.0
id1 = id2 + t1
```

6. Phase 6: Code Generation (Target Assembly):

Emits target machine instructions using hardware CPU registers (`R1`, `R2`):

```
LDF  R2, id3      ; Load float rate into R2
MULF R2, #60.0    ; Multiply R2 by float literal 60.0
LDF  R1, id2      ; Load float initial into R1
ADDF R1, R2       ; Add R2 to R1
STF  id1, R1      ; Store result into memory position
```

Topic 4 & 5: Lexical Analysis & Input Buffering

1. Fundamental Terminology:

Token: An abstract terminal category returned to the parser (e.g., ``id``, ``num``, ``if``, ``while``, ``assign_op``).

Pattern: A formal grammatical specification (regular expression) describing the set of strings representing a token (e.g., `[a-zA-Z_][a-zA-Z0-9_]*`).

Lexeme: The concrete sequence of source characters matching the pattern (e.g., ``counter_variable``, ``1024``, ``==``).

2. Input Buffering (Two-Buffer Scheme with Sentinels):

Reading one character at a time using direct system calls incurs massive operating system overhead. A **Two-Buffer Scheme** uses two N -byte blocks (typically $N = 4096$ bytes matching disk blocks) managed by two pointers:

``lexemeBegin`` Pointer: Marks the beginning of the current token being recognized.

``forward`` Pointer: Advances ahead character by character until a token pattern boundary is identified.

Sentinels (``EOF``): Appending an ``EOF`` marker at the end of each buffer half avoids making two comparisons per character (one for buffer boundary and one for character matching), accelerating lexer throughput by over 200%.

Topic 7: Thompson's Construction Algorithm (RE $\rightarrow$ NFA)

Thompson's Construction converts any Regular Expression r into an equivalent Non-Deterministic Finite Automaton ($N(r)$) with ϵ -transitions in linear time $O(|r|)$:

RE Operation	Thompson's Structural Composition	Key Structural Invariants
1. Basic Symbol (a)	Start $\xrightarrow{a}$ Accept (2 states, 1 transition)	Exactly 1 start state and 1 accept state.
2. Concatenation ($r_1 r_2$)	Accept state of $N(r_1)$ merged with start state of $N(r_2)$.	Flows directly from $N(r_1)$ into $N(r_2)$.
3. Union / Alternation ($r_1 \mid r_2$)	New start state branching via ϵ to $N(r_1)$ and $N(r_2)$; both accept states transition via ϵ to a new single accept state.	4 new ϵ -transitions introduced.
4. Kleene Closure (r^*)	New start state branches via ϵ to $N(r)$ and bypasses $N(r)$ to accept state; accept state loops back via ϵ to start of $N(r)$.	Handles zero or multiple repetitions.

Topic 8: Subset Construction Algorithm (NFA $\rightarrow$ DFA)

A Deterministic Finite Automaton (DFA) contains no ϵ -transitions and has at most one outgoing transition per input symbol from any state.

Core Operations in Subset Construction

- $\epsilon\text{-closure}(s)$: Set of all NFA states reachable from state s taking only ϵ -transitions.
- $\epsilon\text{-closure}(T)$: $\bigcup_{s \in T} \epsilon\text{-closure}(s)$ for a set of states T .
- $\text{move}(T, a)$: Set of all NFA states reachable from any state in T on input symbol a .



Step-by-Step Solved Problem: Convert $(a \mid b)^*abb$ to DFA via Subset Construction

NFA States: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$. Accept state: $\{10\}$.

Step 1: Compute Initial DFA State $A = \epsilon\text{-closure}(0) = \{0, 1, 2, 4, 7\}$.

Step 2: Compute Transitions for DFA States (D -Tran Table):

DFA State	NFA State Subset	Input 'a'	Input 'b'
A (Start)	$\{0, 1, 2, 4, 7\}$	$B = \{1, 2, 3, 4, 6, 7, 8\}$	$C = \{1, 2, 4, 5, 6, 7\}$
B	$\{1, 2, 3, 4, 6, 7, 8\}$	$B = \{1, 2, 3, 4, 6, 7, 8\}$	$D = \{1, 2, 4, 5, 6, 7, 9\}$
C	$\{1, 2, 4, 5, 6, 7\}$	$B = \{1, 2, 3, 4, 6, 7, 8\}$	$C = \{1, 2, 4, 5, 6, 7\}$
D	$\{1, 2, 4, 5, 6, 7, 9\}$	$B = \{1, 2, 3, 4, 6, 7, 8\}$	$E = \{1, 2, 4, 5, 6, 7, 10\}$
E* (Accept)	$\{1, 2, 4, 5, 6, 7, 10\}$	$B = \{1, 2, 3, 4, 6, 7, 8\}$	$C = \{1, 2, 4, 5, 6, 7\}$

Topic 9: Hopcroft's DFA Minimization Algorithm

Hopcroft's Algorithm finds the unique minimal DFA by iteratively partitioning the set of all DFA states S into disjoint equivalence classes based on behavioral distinguishability:

Initial Partition (P_0): Divide states into two groups: Accept states (F) and Non-Accept states ($S \setminus F$).

$$P_0 = \{F, S \setminus F\}$$

Refinement Step: For each group $G \in P$ and each input symbol $a \in \Sigma$, if $\text{move}(s, a)$ for states in G leads into different partitions, split G into sub-partitions.

Termination: Repeat refinement until no partition can be further split ($P_{k+1} = P_k$).

State Compression: Replace each final partition group with a single representative state in the minimal DFA.



Top BIT Mesra Exam Questions & Answers (Module I)

Q1. Explain the role of Lexical Analyzer and why it is separated from the Parser. (8 Marks)

1. **Simplicity of Compiler Design:** Separating lexical scanning from grammatical parsing simplifies both phases. The parser deals with clean token streams rather than raw character-level whitespace, newlines, and comments.
2. **Compiler Efficiency:** Specialized high-speed buffering techniques can be applied to the lexical scanner. Over 70% of compilation time is spent reading source characters.
3. **Portability:** Character-set idiosyncrasies (ASCII, UTF-8, Unicode) are isolated strictly inside the lexical phase, making the parser platform-independent.

Q2. State the difference between Compiler and Interpreter across 5 engineering parameters. (6 Marks)

1. **Translation Mechanism:** Compiler translates entire source code into native machine code before execution; Interpreter translates and executes line-by-line at runtime.
2. **Execution Speed:** Compiled machine code executes 10–50× faster than interpreted bytecode.
3. **Memory Usage:** Compiler requires memory for object code storage; Interpreter requires memory for runtime engine.
4. **Error Reporting:** Compiler reports all syntax/semantic errors collectively during compilation; Interpreter stops at the first runtime error encountered.
5. **Examples:** C, C++, Rust (Compiled); Python, Ruby, PHP (Interpreted).

Q3. Describe the structure and working of Lex/Flex tool. (6 Marks)

A Lex program consists of 3 distinct sections separated by `%%` delimiters:

1. **Definitions Section:** Header includes, declarations, and regular definitions (e.g., `digit [0–9]`).
 2. **Rules Section:** Pairs of regular expressions and corresponding C action code (e.g., `{digit}+ { return NUM; }`).
 3. **User Subroutines Section:** Auxiliary C functions including `main()` and `yywrap()`.
- Lex compiles the specification into a fast transition table-driven DFA scanner named `yylex()`.

☰ Module II: Syntax Analysis & Parsing Algorithms — Complete Syllabus Topics

- | | |
|--|---|
| 1. Role of the Parser & Context-Free Grammars (CFG) | 2. Ambiguity Elimination & Grammar Transformations |
| 3. Left Recursion Elimination (Immediate & Indirect Algorithms) | 4. Left Factoring Algorithm for Common Prefixes |
| 5. FIRST and FOLLOW Sets Formal Formulations & Step-by-Step Traces | 6. LL(1) Top-Down Predictive Parser Table & Stack Parsing Walkthrough |
| 7. Bottom-Up Shift-Reduce Parsing & Handle Pruning | 8. LR(0) Canonical Collection of Items & SLR(1) Parsing Tables |
| 9. Canonical LR (CLR(1)) with Lookahead & LALR(1) State Merging | 10. Parsing Conflicts (Shift/Reduce, Reduce/Reduce) & Error Recovery |

Topic 1 & 2: Context-Free Grammars & Ambiguity

A **Context-Free Grammar (CFG)** is formally defined as a 4-tuple $G = (V, T, P, S)$:

V : Finite set of Non-Terminal characters (variables representing syntactic categories).

T : Finite set of Terminal characters (tokens returned by lexical analyzer).

P : Finite set of Production rules of the form $A \rightarrow \alpha$, where $A \in V$ and $\alpha \in (V \cup T)^*$.

$S \in V$: The designated Start Symbol.

Grammar Ambiguity:

A grammar G is **ambiguous** if there exists at least one string $w \in L(G)$ that has two or more distinct parse trees (or equivalently, two distinct leftmost derivations).

Classic example: The Dangling-Else Grammar:

$$\text{stmt} \rightarrow \text{if expr then stmt} \mid \text{if expr then stmt else stmt} \mid \text{other}$$

Resolved by disambiguating the grammar into `matched\_stmt` and `unmatched\_stmt`, matching each `else` with the closest unmatched `then`.

Topic 3 & 4: Grammar Transformations (Left Recursion & Left Factoring)

1. Immediate Left Recursion Elimination:

A production $A \rightarrow A\alpha \mid \beta$ (where β does not start with A) causes top-down parsers to loop infinitely. It is eliminated by introducing a new non-terminal A' :

$$A \rightarrow \beta A', \quad A' \rightarrow \alpha A' \mid \epsilon$$

2. General Left Recursion Elimination Algorithm:

Arrange non-terminals in some arbitrary ordering: $A_1, A_2, \dots, A_n$.

For $i = 1$ to n :

For $j = 1$ to $i - 1$: Replace each production $A_i \rightarrow A_j \gamma$ with $A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$, where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$ are the current A_j productions.

Eliminate immediate left recursion among the A_i productions.

3. Left Factoring Algorithm (Removing Common Prefixes):

When two productions for A share a common prefix ($A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$), the parser cannot determine which branch to choose based on one lookahead token. Left factoring extracts the common prefix:

$$A \rightarrow \alpha A', \quad A' \rightarrow \beta_1 \mid \beta_2$$

Topic 5: Formal Formulations of FIRST and FOLLOW

Formal Mathematical Definitions

FIRST(α): Set of all terminals that begin strings derived from α . If $\alpha \xrightarrow{*} \epsilon$, then $\epsilon \in \text{FIRST}(\alpha)$.

FOLLOW(A): Set of all terminals a that can appear immediately to the right of non-terminal A in some sentential form ($S \xrightarrow{*} \alpha A a \beta$). If A is the rightmost symbol in a derivation, the input end-marker $\$$ is in **FOLLOW(A)**.

Rules for Computing FIRST:

If X is a terminal, **FIRST(X)** = $\{X\}$.

If $X \rightarrow \epsilon$ is a production, add ϵ to **FIRST(X)**.

If $X \rightarrow Y_1 Y_2 \dots Y_k$ is a production:

Add all non- ϵ symbols from **FIRST(Y_1)** to **FIRST(X)**.

If $\epsilon \in \text{FIRST}(Y_1)$, add all non- ϵ symbols from **FIRST(Y_2)** to **FIRST(X)**, and so forth, until some Y_i does not contain ϵ .

If all $Y_1, \dots, Y_k$ contain ϵ , add ϵ to **FIRST(X)**.

Rules for Computing FOLLOW:

Place $\$$ in **FOLLOW(S)**, where S is the start symbol.

If there is a production $A \rightarrow \alpha B \beta$, everything in **FIRST(β)** except ϵ is in **FOLLOW(B)**.

If there is a production $A \rightarrow \alpha B$, or $A \rightarrow \alpha B \beta$ where **FIRST(β)** contains ϵ , everything in **FOLLOW(A)** is in **FOLLOW(B)**.



Step-by-Step Solved Problem: Arithmetic Grammar FIRST & FOLLOW

Grammar (G):

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \epsilon$
- $F \rightarrow (E) \mid \text{id}$

Computed Sets:

Non-Terminal	FIRST Set	FOLLOW Set
E	{(, id}	{), \$}
E'	{+, ϵ }	{), \$}
T	{(, id}	{+,), \$}
T'	{*, ϵ }	{+,), \$}
F	{(, id}	{*, +,), \$}

Topic 6: Top-Down LL(1) Predictive Parsing

An **LL(1) Parser** scans input from Left-to-right, produces a Leftmost derivation, using 1 lookahead token.

LL(1) Grammatical Condition

A grammar G is $LL(1)$ if and only if for every pair of productions $A \rightarrow \alpha \mid \beta$:

1. $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$.
2. If $\epsilon \in \text{FIRST}(\alpha)$, then $\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$.

Complete LL(1) Parsing Table:

Non-Terminal	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

Topics 7 – 9: Bottom-Up LR Parsing Landscape

LR Parsers (Left-to-right scan, Rightmost derivation in reverse) are the most powerful deterministic shift-reduce parsing engines.

Parser Class	Item Structure	Reduce Action Decision	State Complexity & Power
1. LR(0)	$[A \rightarrow \alpha \cdot \beta]$	Reduce across all columns unconditionally.	Fewest states; weak; fails on almost all programming grammars.
2. SLR(1)	$[A \rightarrow \alpha \cdot \beta]$	Reduce $A \rightarrow \alpha$ only on input symbols $a \in \text{FOLLOW}(A)$.	Same number of states as LR(0); resolves many Shift/Reduce conflicts.
3. CLR(1)	$[A \rightarrow \alpha \cdot \beta, a]$ (with Lookahead a)	Reduce $A \rightarrow \alpha$ only on exact lookahead symbol a .	Most powerful deterministic class; massive number of states (thousands).
4. LALR(1)	Merged LR(1) items with identical core $LR(0)$ components.	Reduce on merged lookahead sets.	Same number of states as SLR(1); nearly the parsing power of CLR(1); used in Yacc/Bison.

Top BIT Mesra Exam Questions & Answers (Module II)

Q1. Prove why an ambiguous grammar can never be LL(1) or LR(1). (8 Marks)

Proof: An ambiguous grammar generates at least two distinct parse trees for some string w .

1. In an **LL(1) parser**, this ambiguity manifests as multiple production entries in the same parsing table cell $M[A, a]$, violating the single-entry requirement of deterministic LL(1) tables.
2. In an **LR(1) parser**, the two distinct derivations create either a **Shift/Reduce conflict** (where the parser cannot decide whether to shift or reduce) or a **Reduce/Reduce conflict** (where the parser cannot decide which of two distinct productions to reduce by). Thus, ambiguous grammars can never be parsed deterministically by LL(1) or LR(1).

Q2. Explain the difference between SLR(1) and LALR(1) parsers. (6 Marks)

Both SLR(1) and LALR(1) parsers have the exact same number of states (identical to the LR(0) state machine). However, they differ in how they decide reduction actions:

- **SLR(1):** Places reduce actions for $A \rightarrow \alpha$ in all columns corresponding to $\text{FOLLOW}(A)$. $\text{FOLLOW}(A)$ is a global set that includes symbols that may never legally follow A in that specific context.
- **LALR(1):** Computes specific context-sensitive lookahead sets during item propagation, placing reduce actions only on valid lookaheads. As a result, LALR(1) eliminates many false Shift/Reduce and Reduce/Reduce conflicts present in SLR(1).

☰ Module III: Semantic Analysis & Type Checking — Complete Syllabus Topics

1. Syntax-Directed Definitions (SDD) & Semantic Rules
2. Synthesized Attributes vs. Inherited Attributes
3. S-Attributed Definitions & Bottom-Up Evaluation
4. L-Attributed Definitions & Top-Down Evaluation Orders
5. Dependency Graphs & Topological Sorting for Attribute Evaluation
6. Syntax-Directed Translation Schemes (SDTS) & Action Placement
7. Static Type Checking & Type Systems
8. Type Expressions, Type Equivalence (Structural vs Name)
9. Type Conversions (Implicit Coercion vs Explicit Casting)
10. Symbol Table Organizations (Hash Tables with Scope Chaining)

Topic 1 & 2: Syntax-Directed Definitions (SDD) & Attributes

A **Syntax-Directed Definition (SDD)** is a context-free grammar together with attributes and semantic rules. Attributes are associated with grammar symbols, and rules are associated with productions.

Attribute Class	Definition & Value Flow Direction	Key Evaluation Properties
1. Synthesized Attribute	Computed from the attribute values of the node's children in the parse tree (or lexical values). $A.s = f(Y_1.a_1, Y_2.a_2, \dots, Y_k.a_k)$	Flows strictly Bottom-Up ; naturally evaluated during LR parsing.
2. Inherited Attribute	Computed from the attribute values of the node's parent and/or siblings. $Y_j.i = f(A.a, Y_1.a_1, \dots, Y_{j-1}.a_{j-1})$	Flows Top-Down and Left-to-Right ; passes context (e.g., data type declarations) downward.

Topic 3 & 4: S-Attributed vs. L-Attributed SDD

Structural Classifications

- **S-Attributed SDD:** An SDD that uses **only synthesized attributes**. Can be evaluated during bottom-up parsing (e.g., LR parser) by maintaining an attribute stack in parallel with the parse stack.
- **L-Attributed SDD:** An SDD where every attribute is either synthesized, or inherited with the restriction that for production $A \rightarrow X_1 X_2 \dots X_n$, an inherited attribute of X_j depends only on:
 1. Attributes of the parent A .
 2. Attributes of symbols to the left of X_j ($X_1, X_2, \dots, X_{j-1}$).Every S-Attributed SDD is strictly an L-Attributed SDD ($S\text{-Attributed} \subset L\text{-Attributed}$).

Step-by-Step Solved Problem: Desktop Calculator S-Attributed SDD

Production	Semantic Rule
$L \rightarrow E \mathbf{n}$	$\text{print}(E.\text{val})$
$E \rightarrow E_1 + T$	$E.\text{val} = E_1.\text{val} + T.\text{val}$
$E \rightarrow T$	$E.\text{val} = T.\text{val}$
$T \rightarrow T_1 * F$	$T.\text{val} = T_1.\text{val} * F.\text{val}$
$T \rightarrow F$	$T.\text{val} = F.\text{val}$
$F \rightarrow (E)$	$F.\text{val} = E.\text{val}$
$F \rightarrow \mathbf{digit}$	$F.\text{val} = \mathbf{digit}.\text{lexval}$

Evaluates $(3 + 4) * 5\mathbf{n} \implies 35$ bottom-up in a single pass.

Topic 7 & 8: Type Checking & Type Systems

A **Type Checker** verifies that the type of a construct matches that expected by its context (e.g., the modulo operator `%` requires integer operands in C).

1. Structural vs. Name Equivalence:

Name Equivalence: Two types are equivalent if and only if they share the exact same type name identifier (Strict, fast to check).

Structural Equivalence: Two types are equivalent if they have identical internal structures (e.g., record fields with identical names and types in the same order).

2. Type Conversions:

Implicit Coercion: Automatically performed by compiler (e.g., widening an `int` to `float` during addition `2 + 3.5`).

Explicit Casting: Programmed explicitly by the developer (e.g., `(int)x`).

Topic 10: Symbol Table Organization

The **Symbol Table** is a critical data structure containing record entries for every user-defined identifier in the source program, storing name, type, memory offset, scope level, and parameter signatures.

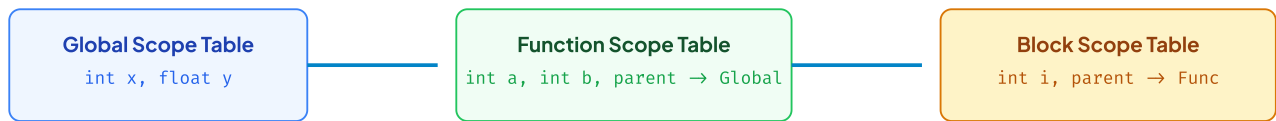


Figure 3.1: Hierarchical Block-Structured Scope Chaining



Top BIT Mesra Exam Questions & Answers (Module III)

Q1. Differentiate between S-Attributed and L-Attributed SDD with syntax-directed rules. (8 Marks)

1. **S-Attributed SDD:** Uses only synthesized attributes. Can be evaluated during bottom-up parsing using an attribute stack without creating an explicit parse tree.
2. **L-Attributed SDD:** Allows both synthesized and inherited attributes, but inherited attributes of symbol X_j in $A \rightarrow X_1 X_2 \dots X_n$ are restricted to depend only on attributes of parent A and left siblings $X_1, \dots, X_{j-1}$. Can be evaluated in a single depth-first, left-to-right parse tree traversal.

Q2. How is dynamic block scoping implemented in a symbol table? (6 Marks)

Block scoping is implemented using a **Stack of Hash Tables** or a chained tree of symbol tables. When entering a new block `{`, a new empty hash table is created and pushed onto the scope stack, with a pointer pointing to its enclosing parent scope table. Symbol lookups search from the current top-of-stack table upwards along parent pointers. When exiting a block `}`, its table is popped and deallocated.

☰ Module IV: Intermediate Code Generation & Runtime Environments — Complete Topics

1. Intermediate Representations (Syntax Trees, DAG, Postfix, TAC)
2. Three-Address Code (TAC) Representations: Quadruples, Triples, Indirect Triples
3. Translation of Expressions & Type Coercion TAC
4. Multi-Dimensional Array Addressing (Row-Major vs Column-Major Proofs)
5. Boolean Expressions Translation & Short-Circuit Evaluation
6. Backpatching Techniques (`makelist`, `merge`, `backpatch`)
7. Runtime Storage Organization & Activation Records (Stack Frames)
8. Storage Allocation Strategies (Static, Stack, Heap)
9. Access Links & Displays for Nested Block Scopes
10. Parameter Passing Mechanisms & Garbage Collection

Topic 1 & 2: Three-Address Code (TAC) Representations

Three-Address Code (TAC) is a linear intermediate representation where each instruction has at most one operator on the right-hand side:

$$x = y \text{ op } z$$

Representation	Internal Data Structure & Fields	Key Advantages & Tradeoffs
1. Quadruples	Four fields: `(op, arg1, arg2, result)`. Explicit temporary names (e.g., `t1`, `t2`) are stored in `result`.	Easy to reorder during code optimization; requires extra memory for temporary names.
2. Triples	Three fields: `(op, arg1, arg2)`. Results of operations are referred to by their instruction index position (e.g., `(0)`, `(1)`).	Space efficient; moving or inserting instructions requires updating all index references.
3. Indirect Triples	Uses an array of pointers to separate triple structures.	Optimizers can reorder instructions by simply swapping pointers in the array without touching the underlying triples.

Step-by-Step Solved Problem: Quadruples, Triples & Indirect Triples for `a = b \* - c + b \* - c`

Step 1: Generate Three-Address Code:

```
(0) t1 = uminus c
(1) t2 = b * t1
(2) t3 = uminus c
(3) t4 = b * t3
(4) t5 = t2 + t4
(5) a = t5
```

Step 2: Quadruple Representation:

#	op	arg1	arg2	result
0	uminus	c	-	t1
1	*	b	t1	t2
2	uminus	c	-	t3
3	*	b	t3	t4
4	+	t2	t4	t5
5	=	t5	-	a

Step 3: Triple Representation:

#	op	arg1	arg2
(0)	uminus	c	-
(1)	*	b	(0)
(2)	uminus	c	-
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	assign	a	(4)

Topic 4: Multi-Dimensional Array Addressing Formulations

1. 1D Array Address Formula:

$$\text{Address}(A[i]) = \text{base} + (i - \text{low}) \times w$$

Where base is the start memory address, low is lower index bound (typically 0 or 1), and w is byte width per element.

2. 2D Array Address Formula (Row-Major Order — C / C++ / Java):

$$\text{Address}(A[i_1, i_2]) = \text{base} + \left((i_1 - \text{low}_1) \times n_2 + (i_2 - \text{low}_2) \right) \times w$$

Where $n_2 = \text{high}_2 - \text{low}_2 + 1$ is the number of elements in each row.

3. 2D Array Address Formula (Column-Major Order — FORTRAN / MATLAB):

$$\text{Address}(A[i_1, i_2]) = \text{base} + \left((i_2 - \text{low}_2) \times n_1 + (i_1 - \text{low}_1) \right) \times w$$

Where $n_1 = \text{high}_1 - \text{low}_1 + 1$ is the number of elements in each column.

Topic 5 & 6: Backpatching in Boolean Expressions

Backpatching is a single-pass technique for generating intermediate code for control flow and boolean expressions without leaving unresolved forward jump target addresses.

`makelist( $i$ )`: Creates a new list containing only jump instruction index i .

`merge( $p_1$ ,  $p_2$ )`: Concatenates two jump target lists p_1 and p_2 .

`backpatch( $p$ ,  $i$ )`: Inserts target instruction label i as the jump destination into all instructions indexed in list p .

Topics 7 – 9: Runtime Storage Organization & Activation Records

During program execution, the operating system allocates a contiguous block of memory divided into 4 major logical segments:

Code Segment (Text): Read-only region holding compiled target machine instructions.

Static / Global Segment: Holds global variables, static constants, and compiler metadata fixed at compile time.

Call Stack: Grows downward dynamically; stores **Activation Records (Stack Frames)** for function invocations.

Heap: Grows upward dynamically; handles explicit dynamic memory allocation (``malloc``, ``free``, ``new``).

Activation Record Field	Functional Purpose in Procedure Call / Return
Actual Parameters	Arguments passed by the caller to the callee.
Returned Values	Space for return data passed back to caller.
Control Link (Dynamic Link)	Points to caller's activation record; used to restore caller's stack frame on return.
Access Link (Static Link)	Points to activation record of the statically enclosing lexical scope; used to access non-local variables.
Saved Machine Status	Program counter (PC), CPU registers, and status flags saved before function call.
Local Data	Local automatic variables allocated within the function.
Temporaries	Temporary values generated during expression evaluation.



Top BIT Mesra Exam Questions & Answers (Module IV)

Q1. Compare Quadruples, Triples, and Indirect Triples with examples. (8 Marks)

1. **Quadruples:** Explicit 4-field structures `(op, arg1, arg2, res)` where temporary names (`t1`) are explicitly stated. Highly flexible for compiler optimization reordering, but uses more memory.
2. **Triples:** 3-field structures `(op, arg1, arg2)` where intermediate results are referenced by instruction array indices `(0)`. Memory compact, but relocating code requires patching all dependent instruction references.
3. **Indirect Triples:** Stores pointers to triple structures in an array. Allows optimizers to reorder and eliminate code simply by modifying pointer positions without touching underlying triple records.

Q2. Explain Call-by-Value, Call-by-Reference, and Call-by-Name parameter passing. (6 Marks)

- **Call-by-Value:** Actual parameter expression is evaluated and a local copy of the value is passed to the callee. Changes inside the function do not affect the caller's variable.
- **Call-by-Reference:** Memory address (pointer) of the actual parameter is passed. Any modification inside the function directly modifies caller's variable.
- **Call-by-Name (Algol 60):** The parameter is not evaluated upfront; instead, the parameter text is literally substituted into the function body (evaluated lazily via thunks every time it is referenced).

Module V: Code Optimization & Target Code Generation — Complete Syllabus Topics

1. Principal Sources of Optimization (Machine-Independent)
2. Basic Blocks Partitioning Algorithm & Control Flow Graphs (CFG)
3. Directed Acyclic Graphs (DAG) Representation for Basic Blocks
4. Local vs. Global Code Optimization Techniques
5. Loop Optimizations (Loop Invariant Code Motion, Strength Reduction, Unrolling)
6. Dominators, Natural Loops & Reducible Flow Graphs
7. Data-Flow Analysis Equations (Reaching Definitions & Available Expressions)
8. Live Variable Analysis & Dead Code Elimination
9. Target Code Generation Issues & Register Allocation by Graph Coloring
10. Peephole Optimization Techniques (Redundant Load/Store, Algebraic Identities)

Topic 1 & 4: Principal Sources of Code Optimization

Code optimization transforms intermediate or target code to run faster, consume less memory, or use less power, while strictly preserving program semantics.

Optimization Technique	Mechanisms & Transformations	Before → After Example
1. Constant Folding	Evaluating operations on compile-time constants upfront.	<code>`x = 3 + 4 * 2` → `x = 11`</code>
2. Constant Propagation	Replacing variables holding known constant values with the constant.	<code>`pi = 3.14; r = 2; area = pi * r * r` → `area = 3.14 * 2 * 2`</code>
3. Common Subexpression Elimination	Reusing previously computed values instead of re-evaluating identical expressions.	<code>`t1 = a + b; ...; t2 = a + b` → `t2 = t1`</code>
4. Dead Code Elimination	Removing instructions whose results are never read by subsequent code or unreachable blocks.	<code>`if (0) { do_something(); }` → `/* Removed */`</code>
5. Strength Reduction	Replacing expensive operations (e.g., multiplication) with cheaper equivalents (addition, bit-shift).	<code>`x = y * 8` → `x = y << 3`</code>
6. Loop Invariant Code Motion	Hoisting computations that evaluate to the same value in every loop iteration outside the loop header.	<code>`while (i < n) { x = a + b; i++; }` → `x = a + b; while (i < n) { i++; }`</code>

Topic 2: Basic Blocks & Control Flow Graphs (CFG)

Algorithm: Partitioning Three-Address Code into Basic Blocks

Step 1: Identify the Leaders (first instructions of basic blocks):

1. The first Three-Address instruction of the intermediate code is a Leader.
2. Any instruction that is the target of a conditional or unconditional jump (`goto L`) is a Leader.
3. Any instruction that immediately follows a conditional or unconditional jump is a Leader.

Step 2: Construct Basic Blocks:

For each Leader, its Basic Block consists of the Leader and all subsequent instructions up to, but not including, the next Leader or the end of the program.



Step-by-Step Solved Problem: Basic Block Partitioning on Dot Product Code

Three-Address Code:

```
(1) prod = 0           ← Leader (Rule 1: First instruction)
(2) i = 1
(3) t1 = 4 * i         ← Leader (Rule 2: Target of goto (3) in line 9)
(4) t2 = a[t1]
(5) t3 = 4 * i
(6) t4 = b[t3]
(7) t5 = t2 * t4
(8) prod = prod + t5
(9) i = i + 1
(10) if i ≤ 20 goto (3)
(11) print(prod)      ← Leader (Rule 3: Follows conditional jump at line 10)
```

Resulting Basic Blocks:

- **Block B_1** : Instructions (1) to (2).
- **Block B_2 (Loop Body)**: Instructions (3) to (10).
- **Block B_3** : Instruction (11).

Topic 3: Directed Acyclic Graphs (DAG) for Basic Blocks

A **DAG (Directed Acyclic Graph)** is an efficient representation for basic blocks that makes local common subexpression elimination, dead code elimination, and array dependency tracking straightforward:

Leaves correspond to initial values of variables or constants.

Interior nodes correspond to operators.

Nodes have labels attached showing current identifier names having that computed value.

Topic 7 & 8: Data-Flow Analysis Frameworks

Framework	Data-Flow Transfer Equations	Direction & Confluence Operator
1. Reaching Definitions	$\text{OUT}[B] = \text{GEN}[B] \cup (\text{IN}[B] \setminus \text{KILL}[B])$ $\text{IN}[B] = \bigcup_{P \in \text{Pred}(B)} \text{OUT}[P]$	Forward Data-Flow; Union ($\cup$) confluence.
2. Available Expressions	$\text{OUT}[B] = e_ \text{GEN}[B] \cup (\text{IN}[B] \setminus e_ \text{KILL}[B])$ $\text{IN}[B] = \bigcap_{P \in \text{Pred}(B)} \text{OUT}[P]$	Forward Data-Flow; Intersection ($\cap$) confluence.
3. Live Variable Analysis	$\text{IN}[B] = \text{USE}[B] \cup (\text{OUT}[B] \setminus \text{DEF}[B])$ $\text{OUT}[B] = \bigcup_{S \in \text{Succ}(B)} \text{IN}[S]$	Backward Data-Flow; Union ($\cup$) confluence.

Topic 9 & 10: Target Code Generation & Peephole Optimization

1. Register Allocation by Graph Coloring (Chaitin's Algorithm):

Construct the **Interference Graph** where nodes represent variables/temporaries and edges connect variables that are simultaneously live at the same program point.

Find a K -coloring of the graph, where K is the number of available physical hardware CPU registers. If successful, no two interfering variables share the same register.

If the graph cannot be K -colored, select candidate variables to **spill** into RAM stack slots.

2. Peephole Optimization:

A local optimization method that examines a short slide window of target assembly instructions ("peephole") and applies immediate algebraic and redundant instruction eliminations:

Eliminating Redundant Loads and Stores:

```
MOV R0, a
MOV a, R0      ; Redundant - Eliminated!
```

Eliminating Unreachable Code: Instructions immediately following an unconditional jump without a label are deleted.

Algebraic Simplifications: Replacing $x + 0 \rightarrow x, x * 1 \rightarrow x, x * 0 \rightarrow 0$.



Top BIT Mesra Exam Questions & Answers (Module V)

Q1. State the algorithm to identify Basic Blocks and construct a Control Flow Graph. (8 Marks)

1. **Identify Leaders:** (i) First instruction of program, (ii) Target of any jump (`goto`), (iii) Instruction immediately following any jump.
2. **Construct Blocks:** For each leader, its basic block consists of all instructions up to the next leader or program end.
3. **Build Control Flow Graph (CFG):** Nodes are the basic blocks. Directed edges $B_i \rightarrow B_j$ exist if control can transfer from B_i to B_j (via conditional/unconditional jump or sequential fall-through).

Q2. Explain Loop Invariant Code Motion and Strength Reduction with code examples. (8 Marks)

- **Loop Invariant Code Motion:** Moves computations whose operand values do not change across loop iterations outside the loop header.

Example: `for (int i=0; i < 100; i++) { t = i \* 4; }` - **Strength Reduction:** Replaces computationally expensive operations (e.g., multiplication inside loops driven by induction variables) with cheaper operations (addition).

Example: `for (int i=0; i < 100; i++) { t = i \* 4; }` → `t = 0; for (int i=0; i < 100; i++) { ... t += 4; }`